

CSCE-765 HW-2

Asritha Sai Annamreddi

UIN: 939002321

Git repo URL: <https://github.com/asritha-tamu/CSCE-765-HW2/tree/main>

Task 0 - Lab preparation

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ python -m pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives
to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptogra
phy-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel
, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$
```

Task 1 - Demonstrate why encryption alone is insufficient

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original command:
{"action": "READ", "path": "notes.txt"}

Ciphertext:
b34da84b2ee179370e53c39a7704981fed843e36435f56ac789a3a395cc4d2bf440e760c

Relay modification:
Original: b'READ'
Modified: b'WRIT'

XOR mask (READ XOR WRIT):
05170810

Modified ciphertext:
b34da84b2ee179370e53c39f600c881fed843e36435f56ac789a3a395cc4d2bf440e760c

Receiver processes modified command:
{"action": "WRIT", "path": "notes.txt"}

Replay of original ciphertext:
{"action": "READ", "path": "notes.txt"}

Receiver processes replay:
{"action": "READ", "path": "notes.txt"}
```

(1) what design you have made

I implemented a baseline communication design using AES-CTR encryption without a MAC. A sender encrypts the fixed command `{"action": "READ", "path": "notes.txt"}` and sends the ciphertext through a relay to the receiver. The relay does not know the encryption key. I also allowed the same ciphertext to be sent again to demonstrate replay.

(2) what happened in your experiment

Original: READ

Modified: WRIT

Since CTR encryption uses XOR, the relay calculates the XOR difference between these two words and applies that difference to the corresponding ciphertext bytes. The receiver successfully decrypted the modified ciphertext as:

```
{"action": "WRIT", "path": "notes.txt"}
```

Replay:

The original ciphertext was sent again. The receiver decrypted and processed the same command a second time because there was nothing in the design to detect that the ciphertext had already been received.

(3) what is the root cause.

The root cause is that **AES-CTR provides confidentiality but not integrity or authentication.**

There is no MAC/authentication tag that allows the receiver to determine whether the ciphertext was modified.

There is also no sequence number, or nonce-tracking mechanism that allows the receiver to detect that an old valid ciphertext has been replayed.

So the basic conclusion is:

Encryption alone is not equal to Integrity + Replay Protection

Task 2 - Authenticated finite -field Diffie-Hellman setup

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ ls
baseline_ctr.py  ffdhe3072.pem  gateway_key.pem  handshake.py  node_key.pem
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python handshake.py
Loading DH parameters...
Loading RSA keys...

Transcript hash:
954a8123345febb42d2e984fdc0dc9ea5d4da3e3101c6aa4a73c7b7ca9ee7e84
Gateway signature: VERIFIED
Node signature:    VERIFIED
DH shared secret:  MATCHED

Derived values:
K_master   = 31c66264733b0901c8f1cbc1b4157bf8c1706e0c672904fcc009690591832646
K_g2n_enc  = dc99237bd693eca1d7e70fa13414f21adb1ccb351c744a6371a85aa3ce667b31
K_g2n_mac  = 249ff38b2190290dce6a0660c0cff066992d44a76d603dadebf8d2677fd7d64e
K_n2g_enc  = e98824e866b5a952768286fd9035c2827fd7b3fc5432976904a1cda22668c26f
K_n2g_mac  = 2fc69dfd65c7a5447d214e48ed7448b4c44e422f5ad007fb51a082f56f265d13
session_id = 2f3c56688f3c0c35

Handshake successful!
```

Concept	What my implementation demonstrates
Diffie-Hellman	Gateway and node independently derive the same secret
Digital signatures	RSA-PSS authenticates the handshake transcript
Entity authentication	Identities are included and checked
Nonces	Fresh 16-byte values make each session transcript different
Replay resistance	Fresh nonces plus role binding prevent reuse
Forward secrecy	Fresh ephemeral DH values mean long-term RSA keys aren't used to derive the session secret

Concept	What my implementation demonstrates
Key separation	Different HMAC labels create separate encryption/MAC keys

Without signatures, an attacker could intercept the DH exchange and replace the public values with their own values.

The attacker could then establish one DH secret with the gateway and another with the node:

Gateway ← attacker → Node

The attacker could potentially relay/decrypt/re-encrypt traffic between the two sides.

With signatures over the DH public values, identities, and nonces are authenticated, so changing those values causes signature verification to fail.

Task 4 - Adversarial tests and security note

Testcases are written in file test_secured.py

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pytest -v test_security.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collected 8 items

test_security.py::test_valid_handshake_and_bidirectional_messages PASSED [ 12%]
test_security.py::test_modified_ciphertext_is_rejected PASSED [ 25%]
test_security.py::test_modified_authenticated_header_is_rejected PASSED [ 37%]
test_security.py::test_replayed_record_is_rejected PASSED [ 50%]
test_security.py::test_record_reflected_into_opposite_Direction_is_rejected PASSED [ 62%]
test_security.py::test_incorrect_rsa_public_key_is_rejected PASSED [ 75%]
test_security.py::test_invalid_transcript_signature_is_rejected PASSED [ 87%]
test_security.py::test_reflected_handshake_message_is_rejected PASSED [100%]

===== 8 passed in 2.16s =====
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Attacker Model and Non-Goals

In this implementation, we assume an attacker can observe, capture, modify, replay, or send messages between the gateway and the node. The attacker may try to change the encrypted data, modify the header, or send an old message again. However, we assume the attacker does not know the encryption keys or MAC keys shared between the two sides.

The goal of this design is to protect messages while they are being transferred. It provides confidentiality, integrity, and protection against replay attacks. However, it does not protect

against every possible attack. For example, it does not stop an authorized user from sending a harmful command, and it does not protect a system if its keys are stolen.

Why CTR Requires Unique IVs

AES-CTR uses an IV to create a sequence of bytes that is combined with the plaintext to produce ciphertext. The IV must be unique for every message encrypted with the same key.

If the same IV and key are reused, the same byte sequence is generated. This can allow an attacker to compare two ciphertexts and learn information about their plaintexts. In some cases, the attacker may even recover parts of the original messages.

In our implementation, the IV is created using the session ID and sequence number. Since the sequence number increases for every message, a new IV is used for each message in that session. The sender must never reuse a sequence number with the same key.

Why Verify Before Decrypting

When a receiver gets a record, it must verify the HMAC before decrypting the ciphertext. The HMAC checks whether the header, IV, or ciphertext has been changed.

If the receiver decrypts before checking the HMAC, it may process modified or attacker-controlled data. This can cause security problems, especially if the decrypted data is passed to another part of the system.

Our implementation verifies the HMAC first. If verification fails, the record is rejected and no plaintext is returned. This ensures that only authenticated records are decrypted.

Why Use Separate Encryption and MAC Keys

Encryption and MAC have different purposes. Encryption protects the contents of a message, while the MAC checks whether the message has been changed.

Using the same key for both purposes is not a good security practice. Instead, our handshake derives separate encryption and MAC keys. It also uses different keys for gateway-to-node communication and node-to-gateway communication.

This separation reduces the chance that a weakness in one operation will affect the other. It also helps prevent messages from one direction from being accepted as messages in the opposite direction.

Comparison with AES-GCM

Our implementation uses AES-256-CTR for encryption and HMAC-SHA-256 for message authentication. This is called an Encrypt-then-MAC construction because the message is encrypted first and then authenticated.

AES-GCM is an example of an authenticated encryption mode. It provides encryption and integrity together using one operation. It can be simpler to implement because the developer does not need to separately manage encryption and MAC operations.

However, AES-GCM also requires unique nonces. Reusing a nonce with the same key can seriously weaken its security. Our CTR and HMAC design also requires unique IVs, correct key separation, and verification before decryption. Both approaches can be secure when implemented correctly.

Why a Valid Record Can Still Contain a Dangerous Tool Call

A valid protected record only proves that the message was created by someone who has the correct keys and that it was not changed during transmission. It does not prove that the command inside the message is safe.

For example, an authorized user may send a command that deletes important files or accesses sensitive information. The record can have a correct HMAC and still contain a dangerous command.

Therefore, encryption and authentication alone are not enough to ensure safe tool usage. The system must also check whether the requested action is allowed, whether the user has permission, and whether the command is safe to execute.